

Performance review

Repository-wide static review with recommendations ranked by likely value, implementation effort, and visual/correctness risk. This revision assumes the raymarch renderer is removed completely and the raster renderer is the only production path.

Review point	Value
Repository	/home/dad/catingard
Branch	experiment/flat-triangle-wireframe
Reviewed commit	9ec630c (pre-update report baseline)
Review date	6 August 2026
Primary target	Quadro M1000M, about 33 ms / 30 FPS budget
Scope	Raster-only runtime, WGSL, terrain/LOD/streaming, diagnostics, ray-removal cleanup, baker, core types, build profile

Executive verdict: Completely removing the raymarch renderer is a sensible simplification, but it is not itself a raster FPS win because raster is already the default path. It removes an alternate GPU pipeline, ray-only textures, warp/history resources, path branches, controls and parity scaffolding. After that cleanup, the largest remaining cost is still deliberate: fixed global L6 geometry plus a near-duplicate ocean mesh. Start with output-identical raster work that removes unused flat-mode calculations and turns off the exposure pipeline that now defaults off.

This report does not remove the renderer or modify runtime behavior; it documents the raster-only removal case. Performance figures are existing repository evidence, not a fresh GPU benchmark; where archived results predate the current commit, that limitation is stated.

1. Ranked recommendations

The first four items are the best low-risk sequence. They preserve the current visual design and can be tested independently. The remaining items have higher upside but also touch render ordering, mesh density, or resource layout.

Rank	Recommendation	Likely benefit	Effort	Risk
P0-1	Skip unused aerial/fog vertex work in flat-triangle mode	Potentially meaningful GPU vertex saving at L6; exact value unknown	Small	Low
P0-2	Suspend or throttle luminance mips/readback while auto exposure is off	Archived meter cost 0.50-1.15 ms GPU plus CPU poll/readback	Small	Low-medium
P0-3	Replace linear terrain point/edge searches with quadtree lookup	Lower simulation cost and movement spikes	Small-medium	Low
P0-4	Throttle and buffer per-frame JSON exposure logging	Less mutex and file I/O on the render thread	Small	Low
P1-5	Draw the 16-sample atmosphere only on background pixels	Potentially large near the surface where terrain covers much of frame	Medium	Medium
P1-6	Give flat water its own lower-density mesh policy	Avoids much of the current 539k ocean triangles in the L6 orbit sample	Medium	Medium
P1-7	Use a tile array/atlas only if draw-call profiling still points there	Can collapse high tile-batch counts	Medium-high	Medium-high
P2-8	Remove ray-only code/resources and make raster the sole path	Lower startup/resource footprint and maintenance surface; little direct raster FPS change	Medium	Medium
P2-9	Reuse authored-baker scratch buffers and cached neighbor distances	Large offline bake saving; no frame-time effect	Small-medium	Low

What to implement first

Implement P0-1 and P0-2 as separate commits, measuring the same fixed cameras before and after each. Then profile CPU simulation time during a long F4 movement run before deciding whether P0-3 or the GPU-side P1 work is next. P0-4 is safe housekeeping but should not be confused with GPU savings.

What not to change yet

- Do not raise the 256-chunk budget. Existing evidence shows that the budget is load-bearing and that satisfying demand can raise mountain frames from about 38 ms to 59 ms.
- Do not count ray-removal cleanup as a raster frame-time win. Measure raster before/after; the direct benefit is reduced alternate-path resources, branches, startup work and maintenance.
- Do not carry ray-only parity scenarios, F5/F11 controls, ray experiments, or ray-specific metrics into the raster-only binary after removal.
- Do not treat the fixed L6 experiment as an accidental bug. Lowering it is a valid visual/performance choice, not an output-identical optimization.

2. Scope, method, and measured evidence

The review followed the active call chains rather than scanning for generic style issues: frame update and presentation in `main.rs`; terrain selection, streaming, collision truth and draw batching in `terrain.rs`; raster, atmosphere and HDR WGSL; the ray-removal dependency surface; the ETOPO/authored baker; core manifest/tile lookup; scenario assertions; and the current handoff's measured experiments.

Current workload evidence

Evidence	Observed result	Interpretation
Latest HUD-on fixed-L6 orbit sample, manual/1785952608-832998	18.605 ms spatial frame, 53.75 FPS, 256 terrain chunks / 589,824 triangles, 234 ocean chunks / 539,136 triangles, 121 draws	The active experiment can duplicate nearly the complete terrain triangle load with water and can become tile-batch heavy.
Near-field source window	10 km settled frame 28.68 -> 24.06 ms; draws 115 -> 10	Batch/source grouping can matter when ordinary tile fallback produces many groups.
All-land ocean culling	Equal-view mean 32.821 -> 31.403 ms; ocean 256 chunks -> 0-1	Avoiding known-dry ocean work is already implemented and measured at 1.418 ms / 4.3%.
Lifted terrain budget	About 50 ns per triangle across 266-677 chunks; cap lift raised mountain cost toward 59 ms	Geometry, not per-chunk CPU submission, dominated that tested path.
Ray removal case	Current raster remains the default path; the alternate ray path owns separate shader, texture, warp/history and parity machinery	Removal simplifies resource lifetime and maintenance. It should not be sold as a direct raster GPU saving.
Archived GPU timestamp runs	Luminance median about 0.50-1.15 ms in available profiled logs	The hidden meter is not free. These runs are older renderer states and must be refreshed before claiming current savings.

Limitations

- This is a static review supported by existing deterministic measurements. No new GPU benchmark was launched, avoiding an uncontrolled thermal or driver-state comparison.
- GPU stage timings group atmosphere, terrain and ocean into one scene interval. Claims about atmosphere or flat vertex savings are hypotheses to benchmark, not measured millisecond promises. Ray timings are excluded from the revised raster-only plan.
- Manual logs under Fifo often show about 16.67 ms. Use Immediate presentation, an idle machine, identical camera/scenario, and warm settled samples for comparisons.
- The experimental branch differs materially from the normal textured renderer. Recommendations identify when they apply only to flat L6. The removal inventory is an explicit hypothetical; no ray files are deleted by this report update.

3. P0 GPU wins for the active flat-L6 branch

P0-1. Skip terrain aerial/fog computation that flat shading discards

Best current-branch easy win: output-identical in flat-triangle mode and aimed directly at the dense L6 vertex workload.

Evidence

- planet.wgsl:526-649 runs the shared vs_main for all modes. Every terrain vertex computes two-sample aerial in-scatter and distance fog at lines 616-629.
- The flat fragment arm returns flat_triangle_colour at lines 958-959 and does not read the aerial transmittance or in-scatter varyings.
- The fixed-L6 orbit sample submits 589,824 terrain triangles. Even with indexed instancing, the shared 33x33 grid invokes this heavy vertex path for every visible instance.

Recommended change: Add a uniform flat-mode branch in vs_main that supplies identity transmittance / zero in-scatter and bypasses both aerial_perspective_components and terrain_distance_fog_components. A dedicated flat vertex entry point/pipeline is preferable only if the uniform branch does not compile or schedule efficiently. Keep height, detail, collision-consistent displacement, normal calculation, biome/lake ownership, and per-triangle specular unchanged.

Prove it: Run Immediate, fixed L6, with identical orbit, ground-detail, eye-level, and summit poses. Compare GPU scene median and captures pixel-for-pixel or within the existing one-value timing tolerance. Also run the normal final debug mode to prove the standard aerial path is unchanged.

Guardrail: Do not remove displaced_surface_normal in the same change. It currently supplies the flat-interpolated land specular requested by the user, even though most other normal work is fragment geometric.

P0-2. Stop paying the full exposure meter cost while auto exposure is disabled

The application now starts with auto exposure off, but still runs the complete meter, readback and adaptation path every frame.

Evidence

- main.rs:1678-1688 polls readbacks, adapts, writes the exposure uniform and logs an exposure sample every frame.
- main.rs:2289-2294 always encodes luminance extraction/downsampling and a 1x1 readback; main.rs:2420 maps it after submission.
- hdr.rs:581-635 performs one full-resolution extraction pass plus one render pass per mip. At 1280x720 that is 11 mip levels: 1 extraction plus 10 downsample passes.
- hdr.rs:524-545 calls device.poll(Poll) every frame. Archived profiled runs recorded about 0.50-1.15 ms median GPU luminance time, before CPU polling/readback overhead.

Recommended change: When auto exposure is off, either (A) suspend luminance passes/readback/adaptation entirely, or (B) meter at 2-4 Hz if an immediately adapted state on key-6 enable is a hard requirement. Keep the exposure uniform fixed at 1.0. On re-enable, use the last valid meter and existing adaptation rather than snapping. Scenario-authored exposure assertions should explicitly opt into full-rate metering.

Prove it: Profile current HEAD first, then compare GPU luminance, total GPU, submit/present, and visible output in auto-off mode. Toggle key 6 after a bright/dark transition and verify no discontinuity beyond the existing adaptation contract. Run ground_to_orbit, stare_at_sun, and fixed-exposure sunrise/sunset scenarios.

Guardrail: The hidden meter was deliberately kept adapting while fixed exposure was presented. Preserve that diagnostic behavior in scenarios or document the new re-enable warm-up; do not silently alter deterministic exposure tests.

4. P0 CPU and frame-pacing wins

P0-3. Index terrain point queries and edge-neighbor lookup

The current small limits keep these loops bounded, but the code performs repeated linear scans exactly where high-speed flight and topology changes can amplify them.

Evidence

- `terrain.rs:1991-2034` checks 4 edges x 8 samples per active node. Each sample linearly scans the active node slice to find the containing patch. At 256 leaves this is on the order of two million containment tests per update.
- `terrain.rs:884-927` finds a height by scanning every resident tile and choosing the finest match. The cache cap is 384 tiles on current HEAD.
- `terrain.rs:933-1007` scans every concurrently drawn surface node for conservative collision truth, then falls back to another cache scan.
- The movement sweep samples up to 64 substeps at 0.5 m, and clearance is rechecked after destination-frontier selection. Correctness is necessary; repeated lookup cost is not.

Recommended change: Retain the active `BTreeSet` through stitching instead of converting it immediately to a `Vec`. Resolve an edge sample by face/level quadtree ancestry rather than scanning all leaves. For height queries, derive the face/tile key once and walk ancestors through the resident hash map. Add a small active-surface lookup that still returns the maximum across incoming/outgoing transition patches. Memoize the current camera-position query within a frame so clearance and near-plane logic can share it.

Prove it: Add counters/timers around LOD update, stitching, collision sweep and height lookup. Compare F4 idle, slow W, and accelerated W scenarios with 256 chunks. Require identical selected nodes, edge-stitch bits, minimum 30 m moving clearance, idle 2 m clearance, seam delta and screenshots.

Guardrail: The collision query intentionally uses the highest of all concurrently drawn transition surfaces. A simple 'finest tile wins' replacement would be faster but wrong and could reintroduce underground frames.

P0-4. Throttle and buffer interactive JSON tracing

Easy CPU/I/O cleanup: exposure records are emitted every rendered frame to a mutex-protected, unbuffered File even though normal spatial logs are already limited to 2 Hz.

Evidence

- `debug.rs:29-53` wraps `File` directly in `Arc<Mutex<...>>`; no `BufWriter` is used.
- `debug.rs:1059-1075` emits a JSON tracing event for every exposure frame.
- Interactive spatial and optional render-profile samples already use a 0.5 s cadence in `main.rs:1482-1485`, so per-frame exposure logging is the outlier.

Recommended change: Keep assertion tracking at the cadence scenarios require, but log interactive exposure at the existing spatial cadence or only on meaningful change. Wrap the file in `BufWriter` and flush at screenshot, manifest finalization, scenario completion and clean shutdown. Keep errors/warnings immediate if needed.

Prove it: Measure CPU render time and frame-time variance with logging on, not only mean FPS. Confirm JSON remains valid after normal exit and scenario failure, and that exposure sample-count assertions still pass.

Guardrail: Do not remove the scenario evidence stream. Separate 'assert every frame in memory' from 'serialize every frame to disk'.

Lower-priority allocation cleanup

`TerrainRenderer::update` creates several short-lived `Vec/BTreeSet` work lists and sorts candidates every frame. Reusable scratch buffers and deduplication before sorting are sensible only after P0-3 timing shows allocator/sort cost is material. The cache-eviction sort occurs only over capacity and is not a first target.

5. P1 rendering architecture opportunities

P1-5. Render atmosphere only where the depth buffer is still background

The raster path currently runs a 16-sample fullscreen atmospheric march even for pixels immediately overwritten by terrain.

Evidence

- `main.rs:2168-2173` draws atmosphere before terrain in the same pass.
- `atmosphere.rs:40-44` uses depth compare Always and no depth writes.
- `atmosphere.wgsl:368-426` performs 16 scattering samples for every pixel whose atmosphere/planet interval is non-empty.
- Near the surface, terrain can cover most of the frame, making the overdraw ratio much worse than at orbit.

Recommended change: Prototype terrain/ocean first, then a second atmosphere pass using reversed-Z depth Equal against the untouched clear value (or an equivalent background mask). Terrain already applies its own aerial transmittance/in-scatter, so the fullscreen sky need not shade opaque terrain pixels. Preserve sky-only debug mode with a direct background pass.

Prove it: Split scene GPU timestamps into atmosphere and terrain/ocean for the experiment. Compare ground, mountain horizon, coastline, orbit, limb, night, sunrise and sunset captures. Inspect one-pixel silhouettes and holes at terrain/ocean boundaries; the extra pass transition must earn its cost.

Guardrail: Render-order changes can expose edge cracks, depth precision issues, or alter the sun/sky composition contract. This is a benchmark prototype, not an automatic refactor.

P1-6. Decouple flat water tessellation from terrain L6

The current flat-water experiment uses the terrain's 32x32 chunk grid. In the high-orbit sample it submits 539,136 ocean triangles in addition to 589,824 terrain triangles.

Evidence

- `terrain.rs:1800-1880` draws ocean and land batches over the same canonical topologies.
- Known-dry ocean culling is already implemented and measured, so the remaining global/coastal load represents possible-water regions rather than an obvious culling bug.
- Flat mode explicitly disables wave displacement and uses a radial normal. Water no longer needs terrain-equivalent geometric frequency for its shape.

Recommended change: Give the analytic sea shell an independent grid density or fixed lower LOD in flat mode. A 16x16 grid would cut per-ocean-chunk triangles by about 75%; select the actual level from coastline error, not that example. Keep per-fragment height/biome ownership so coasts and retained lakes remain shaped and flat.

Prove it: Measure ocean-heavy orbit, coastline, lake and low flyover poses. Require identical sea level, no raised/angled water, no inland square holes, no shoreline gaps, and unchanged land depth. Compare ocean GPU time and triangle counts separately.

Guardrail: Coastline clipping is fragment-owned. Coarsening the shell is safe only while the mesh still covers the shoreline silhouette and depth interpolation remains stable.

P1-7. Reduce tile bind-group batches only where measurements iustify it

High fixed-L6 orbit can reach 121 draws, but earlier normal-renderer tests held draw calls at 12 across hundreds of chunks. Batching is situational, not the universal bottleneck.

Evidence

- The raster near-field window already proved the leverage in one bad case: draw calls 115 -> 10 and frame time 28.68 -> 24.06 ms at the settled 10 km view.
- Ordinary terrain groups instances by resolved source tile; each group binds a different three-texture tile bind group.
- The current draw loop redundantly rebinds shared vertex/index topology inside each batch, a very small cleanup before a resource-layout rewrite.

Recommended change: First avoid redundant topology buffer bindings across adjacent batches. If GPU/driver traces still correlate frame cost with tile group count, move resident tiles into texture arrays/atlas pages and pass a per-instance layer/page index, collapsing ordinary terrain and ocean groups. Preserve the near-field window as its own resource class.

Prove it: Compare identical views spanning low and high draw counts. Track CPU encode time, GPU scene time, draw calls, tile upload time, cache residency and source fallback. Validate every cube face, gutters, integer biome sampling and eviction/reuse of array layers.

Guardrail: An atlas/array is medium-high risk and touches streaming ownership. Do not undertake it solely because a HUD draw count looks high.

6. Complete raymarch-renderer removal

Assumption for this revision: the foveated ray renderer is removed end-to-end and raster is the only supported render path. This is a cleanup and product-scope decision, not a claim that raster suddenly renders faster. The current app already defaults to raster, so the direct steady-state GPU change should be near zero; the value is fewer resources, branches, shader compiles, controls, tests and failure modes.

Removal inventory

Area	Remove or simplify	Keep
Render-path state	Delete <code>RenderPath::FoveatedRay</code> , F5 toggling, ray labels, path-dependent branches and ray-only HUD fields.	Raster path and its existing debug modes.
Renderer/resources	Remove <code>FoveatedRenderer</code> construction, <code>resize/update/draw</code> calls, ray uniform/face-array textures, warp/history textures, unwarp and ray bind groups/pipelines.	Raster terrain near-field window, shared HDR/post, atmosphere, sun and egui resources.
Shaders/modules	Delete <code>foveated.rs</code> , <code>foveated_debug.wgsl</code> and <code>foveated_unwarp.wgsl</code> only after all call sites/tests are removed.	Raster <code>planet.wgsl</code> , <code>shared_planet.wgsl</code> and <code>atmosphere/HDR</code> shaders.
Diagnostics/scenarios	Remove ray-only timestamp fields, ray near-field coverage, ray-hit debug mode, ray experiment keys, and <code>render_path_parity</code> if it has no raster-only replacement.	Raster probes, haze, terrain seam/clearance assertions and deterministic raster scenarios.
Terrain API	Delete only methods/constants/comments proven ray-exclusive, such as ray surface fallback helpers and ray near-field assumptions.	Raster height breakdown, raster source window and any generic collision query still used by F4 or tests.

Expected performance effect

- Startup and resize should create fewer shader modules, pipelines, texture arrays, bind groups and history targets. Record startup time and GPU memory/resource counts before and after.
- The raster frame should not be credited with the removed ray's 64.05 ms low-pose cost: that path was not executing concurrently. Any raster frame change should be measured separately and is likely limited to branch/state cleanup.
- The ray-specific GPU profiler stage disappears. Remove `raymarch_ms` from logs rather than emitting a permanent zero, and update profile consumers/schema assertions together.

Validation order

- Compile and run the workspace after deleting the path. Search for stale `FoveatedRay`, `foveated`, `raymarch`, `F5/F11` and `render-path-parity` references.
- Run raster orbit, fixed-L6 ground detail, eye-level, summit, ocean/coast, high-speed-clearance, sunrise/sunset, night-side and atmosphere scenarios. Preserve image, seam, clearance, ocean-flatness and exposure assertions.
- Compare raster-only startup, resize, CPU simulation, GPU scene, GPU memory/resource counts, binary size and steady-state frame time. Treat a regression in raster captures as unrelated until isolated; do not restore the removed path as a workaround.

Do not over-delete

The raster near-field source window is separate from the ray renderer and remains valuable. Likewise, generic surface-height and probe code may serve raster collision and diagnostics even if a comment currently mentions the ray path. Remove by call graph, not by matching the word 'ray'.

7. Offline baker, streaming, core types, and build

Authored baker: obvious high-value offline work

The active ETOPO bake skips synthetic erosion, so these changes will not improve current frame time or an ETOPO rebake. They matter for the authored Earth-like generator, whose default is a 4096x2048 grid with 2,048 erosion iterations.

Finding	Evidence	Recommendation
Large scratch allocation every erosion iteration	terrain.rs:106-142 allocates an f64 erosion Vec for all about 8.39 million cells on each of 2,048 iterations (about 64 MiB each time).	Allocate once, overwrite every element, and reuse. Preserve zero for no-downstream cells.
Repeated spherical acos in stable neighborhoods	terrain.rs:128-133 calls grid.distance_meters in every erosion iteration; compute_flow_directions also recalculates candidate distances inside comparisons.	Cache the selected downstream distance for each 32-iteration flow epoch, or precompute the eight row/neighbor distance patterns. Rewrite max selection as one pass so each candidate distance is evaluated once.
Thermal scratch churn	terrain.rs:153-189 allocates outgoing and clones the full height grid every eighth iteration.	Reuse outgoing and old-height buffers or use explicit double buffering.
Sequential tile generation/write	export.rs:46-65 samples, constrains and writes 3,252 tiles serially; write_tile creates a directory and three files per tile.	Parallelize only within a level after parents are complete, with bounded workers. Longer term, consider one packed tile payload/index to reduce filesystem overhead.

Runtime streaming

The runtime uses one background loader thread, opens three raw files per tile, and uploads at most four tiles per frame. That is deliberately bounded and avoids decode cost, but can extend fallback residency. Treat a packed outmap or bounded multi-worker loader as a streaming-latency project, not a steady-FPS easy win. Measure request-to-resident latency, main-thread GPU texture creation/upload time, and disk queue depth first.

Core types

No material runtime problem was found in the small core-types crate. `best_available_ancestor` performs at most 19 binary searches over the sorted manifest list. The renderer already memoizes this in its baked-error walk where it is called thousands of times. Converting the manifest to a hash set would add memory and startup work without evidence that this remaining path matters.

Release profile

The workspace has no custom `[profile.release]`. A benchmark-only trial of `lto = "thin"` and `codegen-units = 1` is cheap and may help CPU-heavy LOD/baker code, but it is unlikely to move a GPU-bound Quadro frame materially and will increase build time. Do not present it as a guaranteed FPS win. `target-cpu=native` should remain local-only because it reduces binary portability.

8. Recommended implementation and measurement plan

Step	Change	Primary metric	Required regression proof
1	Flat vertex aerial/fog bypass	GPU scene ms in fixed-L6 views	Pixel-equivalent flat captures; normal final/aerial modes unchanged
2	Auto-off meter suspend or throttle	GPU luminance ms and total frame ms	Key-6 transition; exposure scenarios; fixed 1.0 presentation
3	Interactive log throttling/buffering	CPU render time and frame-time variance	Valid JSON and complete scenario assertions after clean/failure exits
4	Topology and height-query indices	Simulation ms during long F4 W runs	Same frontier/stitch bits, 30 m moving and 2 m idle clearance, zero new seams
5	Background-only atmosphere prototype	Separate atmosphere and terrain GPU ms	Surface/sky parity across ground, horizon, orbit, twilight and night
6	Independent flat-water mesh	Ocean GPU ms and triangles	Exact sea level, coastline/lake ownership, no holes or raised water
7	Only if still indicated: tile array/atlas	Encode ms, GPU scene ms, draw count	Streaming, gutters, integer biome, cache eviction and cube-face validation
8	Raster-only cleanup audit after ray removal	Startup/resources/binary size and raster frame neutrality	No stale path symbols; raster scenarios and captures remain green

Benchmark protocol

- Build release into a checkout-specific CARGO_TARGET_DIR and run the exact resulting binary.
- Use CATINGARDEN_PRESENT_MODE=immediate. Confirm no other catinthegarden process is running and the GPU is not thermally throttled.
- Use deterministic scenarios or frozen manual poses. Discard cold streaming and first-two-cut samples unless the task is explicitly streaming latency.
- Record median and p90 spatial frame time, GPU stage timings, CPU simulation/encode/submit, chunks, terrain/ocean triangles, draw calls, fallbacks, resident tiles and seam delta.
- Run at least three alternating A/B runs per change. The handoff already shows single-run differences can be unstable.
- Treat captures and geometric probes as acceptance criteria, not optional illustration. A faster frame that changes the surface, coast, exposure, or twilight is a regression.

Suggested pass/fail table

Scenario/pose	Why it is in the matrix
Fixed-L6 6,000 km orbit	Maximum global geometry and tile groups
Landing-site ground detail / eye level	Near terrain, collision truth, fragment fill and atmosphere overdraw
Highest-prominence pose	High slopes, specular, sparse-source fallback
Ocean flyover and coastline	Water density, clipping, lakes and ocean-heavy shader cost
Sunrise, sunset blue hour, limb, night side	Atmosphere ordering and exposure regressions
Manual high-speed clearance	Worst repeated terrain point queries and destination-frontier recheck
Raster-only startup/resize and normal render path	Removed resources/branches without raster regressions

Appendix A. Evidence map

Area	Primary locations reviewed
Frame/update/presentation	crates/app/src/main.rs:1190-1295, 1428-1502, 1645-1718, 1935-2435
Terrain source/cache/LOD/batches	crates/app/src/terrain.rs:430-830, 884-1015, 1318-1885, 1888-2045, 2235-2690
Raster shader	crates/app/src/planet.wgsl:229-329, 430-649, 692-865, 886-1195
Shared terrain/lighting	crates/app/src/shared_planet.wgsl:459-621, 695-770, 1021-1305, 1460-1600
Atmosphere	crates/app/src/atmosphere.rs:1-76; crates/app/src/atmosphere.wgsl:1-450
HDR/exposure	crates/app/src/hdr.rs:330-790, 943-948; crates/app/src/hdr.wgsl
Ray-removal audit	crates/app/src/main.rs ray-path call sites; crates/app/src/foveated.rs; foveated_debug.wgsl; foveated_unwarp.wgsl; render_path_parity scenario
Diagnostics/scenarios	crates/app/src/debug.rs:1-1270; crates/app/src/scenario.rs; crates/app/scenarios
Baker/export	crates/baker/src/terrain.rs, etopo.rs, export.rs, grid.rs, config.rs
Core contract	crates/coretypes/src/lib.rs
Measured history	docs/HANDOFF.md, AGENTS.md, test-runs/**/log.jsonl
Build	Cargo.toml and crate manifests

Appendix B. Review conclusions by subsystem

Subsystem	Conclusion
Active flat raster	Has two strong output-identical targets: unused per-vertex aerial/fog and auto-off luminance work. Geometry remains the main deliberate load.
Normal textured raster	Already contains several measured bailouts and near-field batching work. Avoid assuming flat-mode wins apply unchanged.
Terrain CPU	Bounded but contains avoidable $O(\text{nodes squared})$ and cache scans, most relevant to fast movement/frame pacing.
Atmosphere	High arithmetic intensity with obvious opaque-terrain overdraw; ordering prototype is justified but visually sensitive.
Ocean	Dry culling is good. Flat mode still pays terrain-like geometry in water-heavy views; independent density is the next lever.
Ray removal	A raster-only product scope removes alternate-path resources and maintenance burden; it is not a direct raster FPS claim.
Streaming	Bounded and asynchronous but file-heavy/single-worker. Optimize latency only after request-to-resident measurement.
Baker	Parallel overall, but authored erosion has clear scratch-allocation and repeated-distance costs. ETOPO path does not pay the erosion loop.
Core/build	No obvious hot core-type defect. Thin LTO/codegen trial is optional and low priority.

Bottom line: take the two output-identical GPU wins first, measure them independently, then let the new stage timings decide between CPU indexing, atmosphere masking, and flat-water density. The project already has strong deterministic visual and geometric tests; use them to keep performance work from reopening the rendering bugs that have just been fixed.